

INTRODUCING ANDROID DATABASES

Android provides structured data persistence through a combination of SQLite databases and Content Providers. SQLite databases can be used to store application data using a managed, structured approach. Android offers a full SQLite relational database library. Every application can create its own data-bases over which it has complete control.

Having created your underlying data store, Content Providers offer a generic, well-defined interface for using and sharing data that provides a consistent abstraction from the underlying data source.

SQLite Databases

Using SQLite you can create fully encapsulated relational databases for your applications. Use them to store and manage complex, structured application data. Android databases are stored in the `/data/data/<package_name>/databases` folder on your device (or emulator). All databases are private, accessible only by the application that created them.

Database design is a big topic that deserves more thorough coverage than is possible within this book. It is worth highlighting that standard database best practices still apply in Android. In particular, when you're creating databases for resource-constrained devices (such as mobile phones), it's important to normalize your data to minimize redundancy.

Content Providers

Content Providers provide an interface for publishing and consuming data, based around a simple URI addressing model using the `content://` schema. They enable you to decouple your application layers from the underlying data layers, making your applications data-source agnostic by abstracting the underlying data source.

Content Providers can be shared between applications, queried for results, have their existing records updated or deleted, and have new records added. Any application — with the appropriate permissions — can add, remove, or update data from any other application, including the native Android Content Providers.

Several native Content Providers have been made accessible for access by third-party applications, including the contact manager, media store, and calendar, as described later in this chapter.

By publishing your own Content Providers, you make it possible for you (and other developers) to incorporate and extend your data in new applications.

INTRODUCING SQLITE

SQLite is a well-regarded relational database management system (RDBMS). It is:

- **Open-source**
- **Standards-compliant**
- **Lightweight**
- **Single-tier**

It has been implemented as a compact C library that's included as part of the Android software stack.

By being implemented as a library, rather than running as a separate ongoing process, each SQLite database is an integrated part of the application that created it. This reduces external dependencies, minimizes latency, and simplifies transaction locking and synchronization.

SQLite has a reputation for being extremely reliable and is the database system of choice for many

consumer electronic devices, including many MP3 players and smartphones.

Lightweight and powerful, SQLite differs from many conventional database engines by loosely typing each column, meaning that column values are not required to conform to a single type; instead, each value is typed individually in each row. As a result, type checking isn't necessary when

assigning or extracting values from each column within a row.

CONTENT VALUES AND CURSORS

Content Values are used to insert new rows into tables. Each ContentValues object represents a single table row as a map of column names to values.

Database queries are returned as Cursor objects. Rather than extracting and returning a copy of the result values, Cursors are pointers to the result set within the underlying data. Cursors provide a managed way of controlling your position (row) in the result set of a database query.

The Cursor class includes a number of navigation functions, including, but not limited to, the following:

- moveToFirst — Moves the cursor to the first row in the query result
- moveToNext — Moves the cursor to the next row
- moveToPrevious — Moves the cursor to the previous row
- getCount — Returns the number of rows in the result set
- getColumnIndexOrThrow — Returns the zero-based index for the column with the specified name (throwing an exception if no column exists with that name)
- getColumnName — Returns the name of the specified column index
- getColumnNames — Returns a string array of all the column names in the current Cursor
- moveToPosition — Moves the cursor to the specified row
- getPosition — Returns the current cursor position

Android provides a convenient mechanism to ensure queries are performed asynchronously.

The CursorLoader class and associated Loader Manager (described later in this chapter) were introduced in Android 3.0 (API level 11) and are now also available as part of the support library, allowing you to leverage them while still supporting earlier Android releases.

Later in this chapter you'll learn how to query a database and how to extract specific row/column values from the resulting Cursors.

Service Class

A Service is an application component representing either an application's desire to perform a longer-running operation while not interacting with the user or to supply functionality for other applications to use. Each service class must have a corresponding `<service>` declaration in its package's `AndroidManifest.xml`. Services can be started with `Context.startService()` and `Context.bindService()`.

Note that services, like other application objects, run in the main thread of their hosting process. This means that, if your service is going to do any CPU intensive (such as MP3 playback) or blocking (such as networking) operations, it should spawn its own thread in which to do that work. More information on this can be found in [Processes and Threads](#). The `IntentService` class is available as a standard implementation of Service that has its own thread where it schedules its work to be done.

What is service Class

What is a Service?

Most confusion about the Service class actually revolves around what it is *not*:

- A Service is **not** a separate process. The Service object itself does not imply it is running in its own process; unless otherwise specified, it runs in the same process as the application it is part of.
- A Service is **not** a thread. It is not a means itself to do work off of the main thread (to avoid Application Not Responding errors).

Thus a Service itself is actually very simple, providing two main features:

- A facility for the application to tell the system *about* something it wants to be doing in the background (even when the user is not directly interacting with the application). This corresponds to calls to [Context.startService\(\)](#), which ask the system to schedule work for the service, to be run until the service or someone else explicitly stop it.
- A facility for an application to expose some of its functionality to other applications. This corresponds to calls to [Context.bindService\(\)](#), which allows a long-standing connection to be made to the service in order to interact with it.

When a Service component is actually created, for either of these reasons, all that the system actually does is instantiate the component and call its [onCreate\(\)](#) and any other appropriate callbacks on the main thread. It is up to the Service to implement these with the appropriate behavior, such as creating a secondary thread in which it does its work.

Note that because Service itself is so simple, you can make your interaction with it as simple or complicated as you want: from treating it as a local Java object that you make direct method calls on (as illustrated by [Local Service Sample](#)), to providing a full remoteable interface using AIDL.

Service Lifecycle

There are two reasons that a service can be run by the system. If someone calls [Context.startService\(\)](#) then the system will retrieve the service (creating it and calling its [onCreate\(\)](#) method if needed) and then call its [onStartCommand\(Intent, int, int\)](#) method with the arguments supplied by the client. The service will at this point continue running until [Context.stopService\(\)](#) or [stopSelf\(\)](#) is called. Note that multiple calls to [Context.startService\(\)](#) do not nest (though they do result in multiple corresponding calls to [onStartCommand\(\)](#)), so no matter how many times it is started a service will be stopped once [Context.stopService\(\)](#) or [stopSelf\(\)](#) is called; however, services can use their [stopSelf\(int\)](#) method to ensure the service is not stopped until started intents have been processed.

For started services, there are two additional major modes of operation they can decide to run in, depending on the value they return from [onStartCommand\(\)](#): [START_STICKY](#) is used for services that are explicitly started and stopped as needed, while [START_NOT_STICKY](#) or [START_REDELIVER_INTENT](#) are used for services that should only remain running while

processing any commands sent to them. See the linked documentation for more detail on the semantics.

Clients can also use `Context.bindService()` to obtain a persistent connection to a service. This likewise creates the service if it is not already running (calling `onCreate()` while doing so), but does not call `onStartCommand()`. The client will receive the `IBinder` object that the service returns from its `onBind(Intent)` method, allowing the client to then make calls back to the service. The service will remain running as long as the connection is established (whether or not the client retains a reference on the service's `IBinder`). Usually the `IBinder` returned is for a complex interface that has been [written in aidl](#).

A service can be both started and have connections bound to it. In such a case, the system will keep the service running as long as either it is started *or* there are one or more connections to it with the `Context.BIND_AUTO_CREATE` flag. Once neither of these situations hold, the service's `onDestroy()` method is called and the service is effectively terminated. All cleanup (stopping threads, unregistering receivers) should be complete upon returning from `onDestroy()`.

Process Lifecycle

The Android system will attempt to keep the process hosting a service around as long as the service has been started or has clients bound to it. When running low on memory and needing to kill existing processes, the priority of a process hosting the service will be the higher of the following possibilities:

- If the service is currently executing code in its `onCreate()`, `onStartCommand()`, or `onDestroy()` methods, then the hosting process will be a foreground process to ensure this code can execute without being killed.
- If the service has been started, then its hosting process is considered to be less important than any processes that are currently visible to the user on-screen, but more important than any process not visible. Because only a few processes are generally visible to the user, this means that the service should not be killed except in low memory conditions. However, since the user is not directly aware of a background service, in that state it is considered a valid candidate to kill, and you should be prepared for this to happen. In particular, long-running services will be increasingly likely to kill and are guaranteed to be killed (and restarted if appropriate) if they remain started long enough.
- If there are clients bound to the service, then the service's hosting process is never less important than the most important client. That is, if one of its clients is visible to the user, then the service itself is considered to be visible. The way a client's importance impacts the service's importance can be adjusted through `BIND_ABOVE_CLIENT`, `BIND_ALLOW_OOM_MANAGEMENT`, `BIND_WAIVE_PRIORITY`, `BIND_IMPORTANT`, and `BIND_ADJUST_WITH_ACTIVITY`.
- A started service can use the `startForeground(int, Notification)` API to put the service in a foreground state, where the system considers it to be something the user is actively aware of and thus not a candidate for killing when low on memory. (It is still theoretically possible for the service to be killed under extreme memory pressure from

the current foreground application, but in practice this should not be a concern.)

Note this means that most of the time your service is running, it may be killed by the system if it is under heavy memory pressure. If this happens, the system will later try to restart the service. An important consequence of this is that if you implement [onStartCommand\(\)](#) to schedule work to be done asynchronously or in another thread, then you may want to use [START_FLAG_REDELIVERY](#) to have the system re-deliver an Intent for you so that it does not get lost if your service is killed while processing it.

Other application components running in the same process as the service (such as an [Activity](#)) can, of course, increase the importance of the overall process beyond just the importance of the service itself.

Google Mobile Ads – SDK

To integrate the Google Mobile Ads SDK into a brand new app and use it to display an AdMob banner ad. It should take you about thirty minutes to complete and will give you a good sense of how the SDK functions within an app. If you're new to Google Mobile Ads, this is a great place to start before moving on to more advanced examples.

The ad unit and samples that we provide return test ads. Test ads are always available, even if your account is suspended or disabled. For more information, review the [AdMob policies](#) and learn more about [invalid activity](#)

Prerequisites

- Running Android Studio 1.0 or higher
- Developing for Android level 9 or higher

Steps

Start the new project wizard

Name your project

Set the required SDK version

Add your main activity

Name your activity

Give your app an Ad Unit ID

An [Ad Unit ID](#) is a unique identifier given to the places in your app where ads are displayed. If you have an app with two activities, for example, each displaying a banner, you need two ad units, each with its own ID. AdMob ad unit IDs have the form `ca-app-pub-XXXXXXXXXXXXXXXXX/NNNNNNNNNN`

In order for your new app to display an ad, it needs to include an Ad Unit ID. Open your app's string resource file, which is found at `BannerExample/app/src/main/res/values/strings.xml`.

strings.xml

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <string name="app_name">My Application</string>

    <string name="hello_world">Hello world!</string>

    <string name="action_settings">Settings</string>

    <string name="banner_ad_unit_id">ca-app-pub-
3940256099942544/6300978111</string>

</resources>
```

Add a new `<string>` tag as shown. Note that the Ad Unit ID provided above is just for testing. It allows you to retrieve a sample banner ad and make sure your implementation is correct. You should always use test ads when developing and testing your app--testing with live production ads is a violation of AdMob policy and could cause your account to be suspended. See the [addTestDevice](#) method documentation for information on how to get test ads with your own Ad Unit IDs.

Place an AdView in your main activity layout

There are only two steps remaining before your app is ready to show an ad. First, you need to modify your main activity's layout to include an AdView. Open `BannerExample/app/src/main/res/layout/activity_main.xml` in the editor.

activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools" android:layout_width="match_parent"
    xmlns:ads="http://schemas.android.com/apk/res-auto"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context=".MainActivity">

    <TextView android:text="@string/hello_world" android:layout_width="wrap_content"
    android:layout_height="wrap_content" />

    <com.google.android.gms.ads.AdView

        android:id="@+id/adView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:layout_alignParentBottom="true"
        ads:adUnitId="@string/banner_ad_unit_id"
    </com.google.android.gms.ads.AdView>

</RelativeLayout>
```

Add these to the XML:

1. An additional namespace used for ads:

`http://schemas.android.com/apk/res-auto`

2. A new element for your AdView

You'll be asked to provide `layout_width` and `layout_height`. You can set both to `wrap_content`. In the `AdView` tag, set the `adSize` to `BANNER` and the `adUnitId` to `@string/banner_ad_unit_id`.

Load the ad in the MainActivity class

The last change needed is for you to add to your app's main activity class some Java code that loads an ad into the `AdView`.

Open your `MainActivity.java` file. It's in the `BannerExample/app/src/main/java/` folder, though the exact subdirectory path varies based on the domain you used when creating your project above. Once it's open in the editor, look for the `onCreate` method in the `MainActivity` class

```
protected void onCreate(Bundle savedInstanceState) {  
  
    super.onCreate(savedInstanceState);  
  
    setContentView(R.layout.activity_main);  
  
    AdView mAdView = (AdView) findViewById(R.id.adView);  
  
    AdRequest adRequest = new AdRequest.Builder().build();  
  
    mAdView.loadAd(adRequest);  
  
}
```

Make these two changes:

- 1.Import the `AdRequest` and `AdView` classes.
- 2.Add code to find your `AdView` in the layout, create an `AdRequest`, and then load an ad into the `AdView` with it.

Signing Your Applications

Android requires that all apps be digitally signed with a certificate before they can be installed. Android uses this certificate to identify the author of an app, and the certificate does not need to be signed by a certificate authority. Android apps often use self-signed certificates. The app developer holds the certificate's private key.

Signing Overview

You can sign an app in debug or release mode. You sign your app in debug mode during development and in release mode when you are ready to distribute your app. The Android SDK generates a certificate to sign apps in debug mode. To sign apps in release mode, you need to generate your own certificate.

Signing in Debug Mode

In debug mode, you sign your app with a debug certificate generated by the Android SDK tools. This certificate has a private key with a known password, so you can run and debug your app without typing the password every time you make a change to your project.

Android Studio signs your app in debug mode automatically when you run or debug your project from the IDE.

You can run and debug an app signed in debug mode on the emulator and on devices connected to your development machine through USB, but you cannot distribute an app signed in debug mode.

By default, the *debug* configuration uses a debug keystore, with a known password and a default key with a known password. The debug keystore is located in `$HOME/.android/debug.keystore`, and is created if not present. The debug build type is set to use this debug [SigningConfig](#) automatically.

For more information about how to build and run apps in debug mode, see [Building and Running](#).

Signing in Release Mode

In release mode, you sign your app with your own certificate:

1. *Create a keystore.* A **keystore** is a binary file that contains a set of private keys. You must keep your keystore in a safe and secure place.
2. *Create a private key.* A **private key** represents the entity to be identified with the

app, such as a person or a company.

3. Add the signing configuration to the build file for the `app` module:

```
...
android {
    ...
    defaultConfig { ... }
    signingConfigs {
        release {
            storeFile file("myreleasekey.keystore")
            storePassword "password"
            keyAlias "MyReleaseKey"
            keyPassword "password"
        }
    }
    buildTypes {
        release {
            ...
            signingConfig signingConfigs.release
        }
    }
}
...
```

4. Invoke the `assembleRelease` build task from Android Studio.

The package in `app/build/apk/app-release.apk` is now signed with your release key.

To obtain these passwords from environment variables:

```
storePassword System.getenv("KSTOREPWD")
keyPassword System.getenv("KEYPWD")
```

To have the build process prompt you for these passwords if you are invoking the build from the command line:

```
storePassword System.console().readLine("\nKeystore password: ")
keyPassword System.console().readLine("\nKey password: ")
```

Signing Your App in Android Studio

To sign your app in release mode in Android Studio, follow these steps:

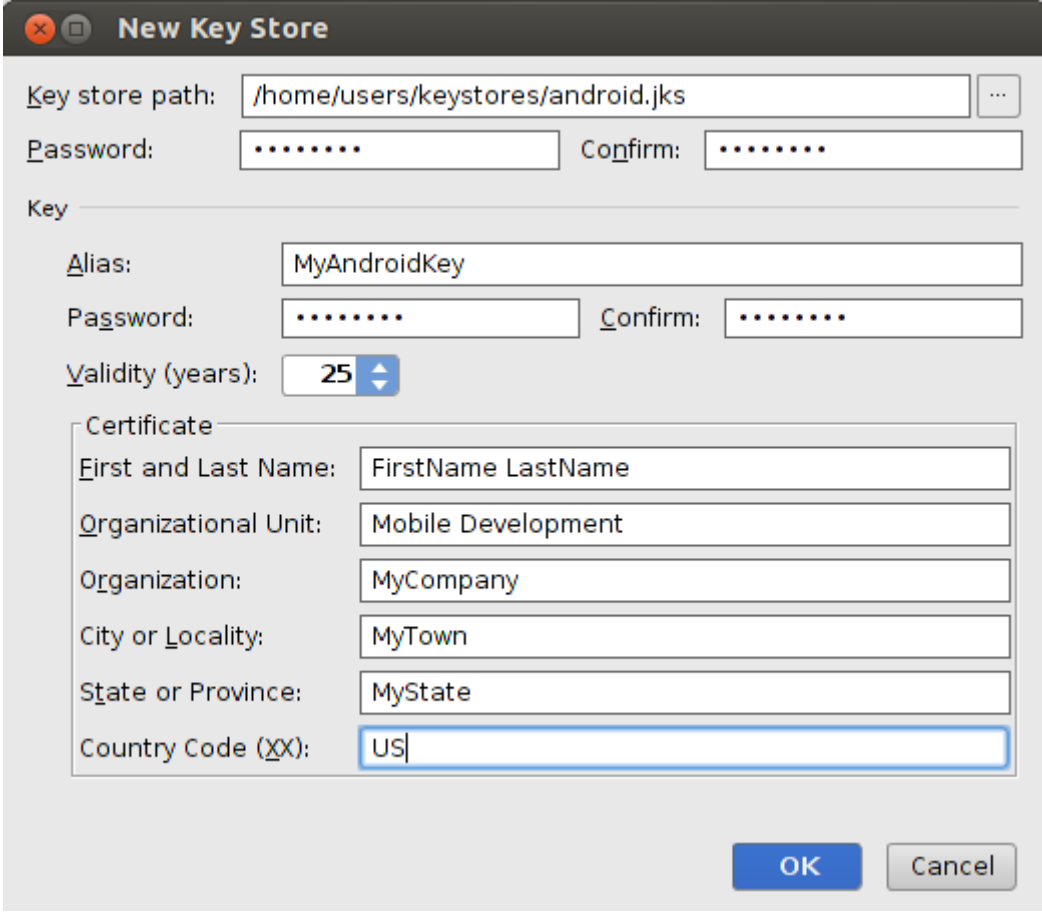
1. On the menu bar, click **Build > Generate Signed APK**.

2. On the *Generate Signed APK Wizard* window, click **Create new** to create a new keystore.

If you already have a keystore, go to step 4.

3. On the *New Key Store* window, provide the required information as shown in figure 1.

Your key should be valid for at least 25 years, so you can sign app updates with the same key through the lifespan of your app.



New Key Store

Key store path: ...

Password: Confirm:

Key

Alias:

Password: Confirm:

Validity (years): ▾

Certificate

First and Last Name:

Organizational Unit:

Organization:

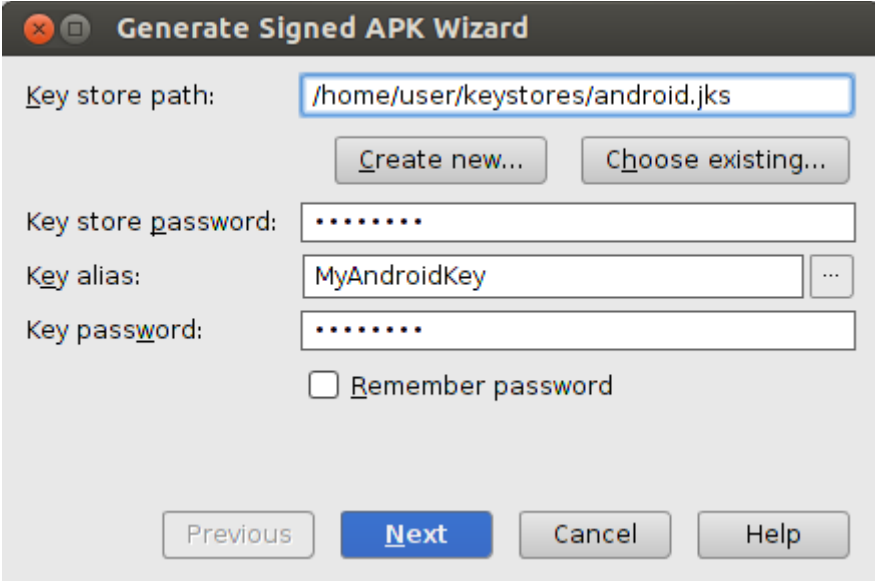
City or Locality:

State or Province:

Country Code (XX):

OK Cancel

4 . On the *Generate Signed APK Wizard* window, select a keystore, a private key, and enter the passwords for both. Then click **Next**.



Generate Signed APK Wizard

Key store path:

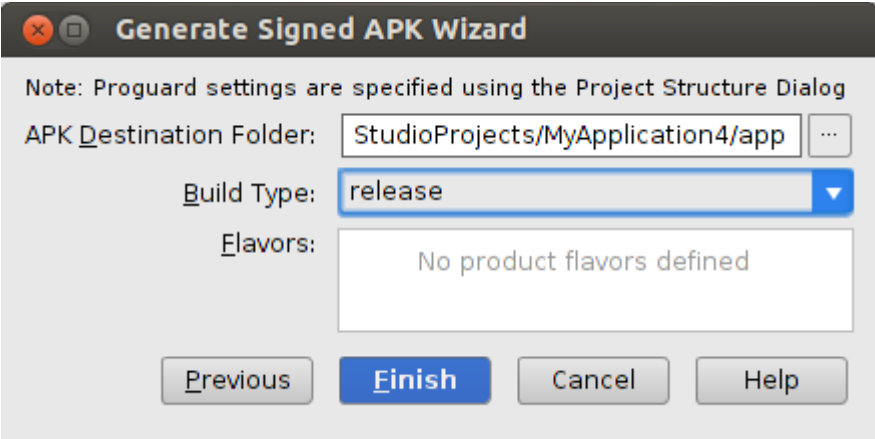
Key store password:

Key alias: ...

Key password:

☐ Remember password

5. On the next window, select a destination for the signed APK and click **Finish**.



Generate Signed APK Wizard

Note: Proguard settings are specified using the Project Structure Dialog

APK Destination Folder: ...

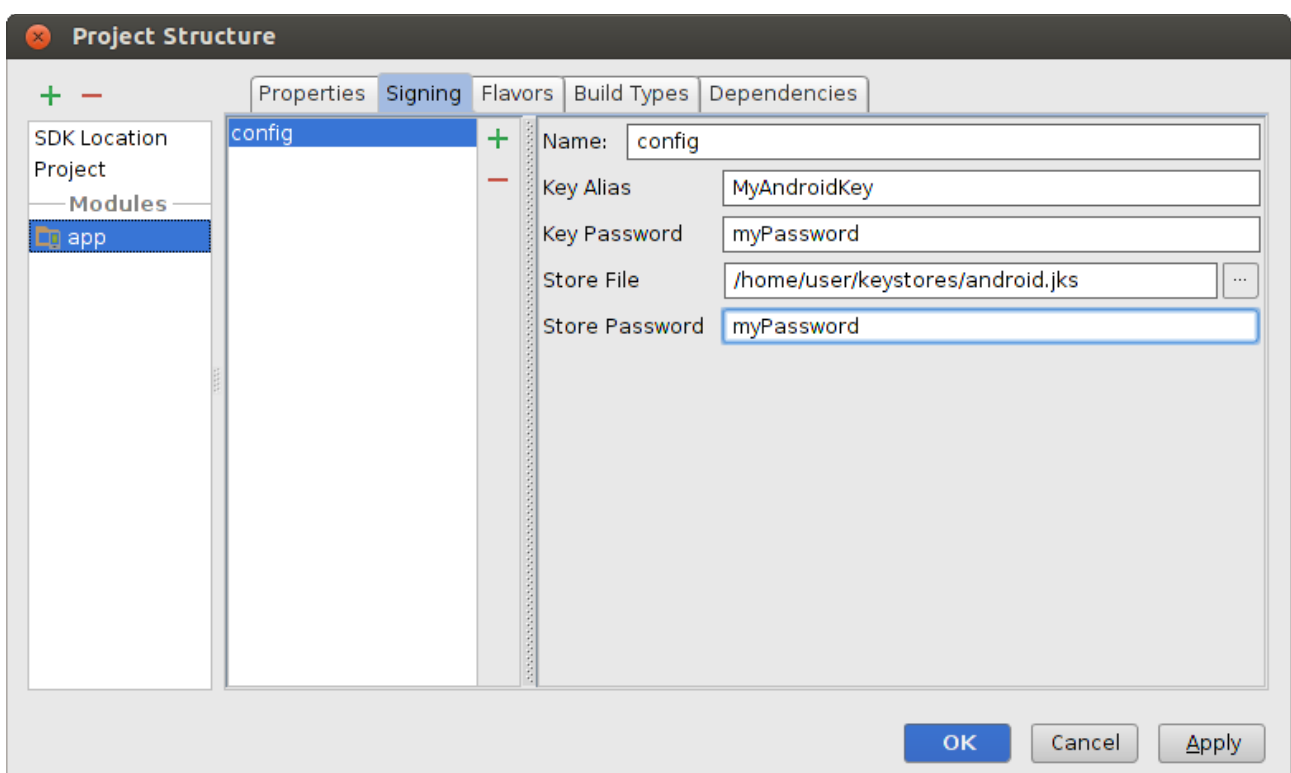
Build Type: ▼

Flavors:

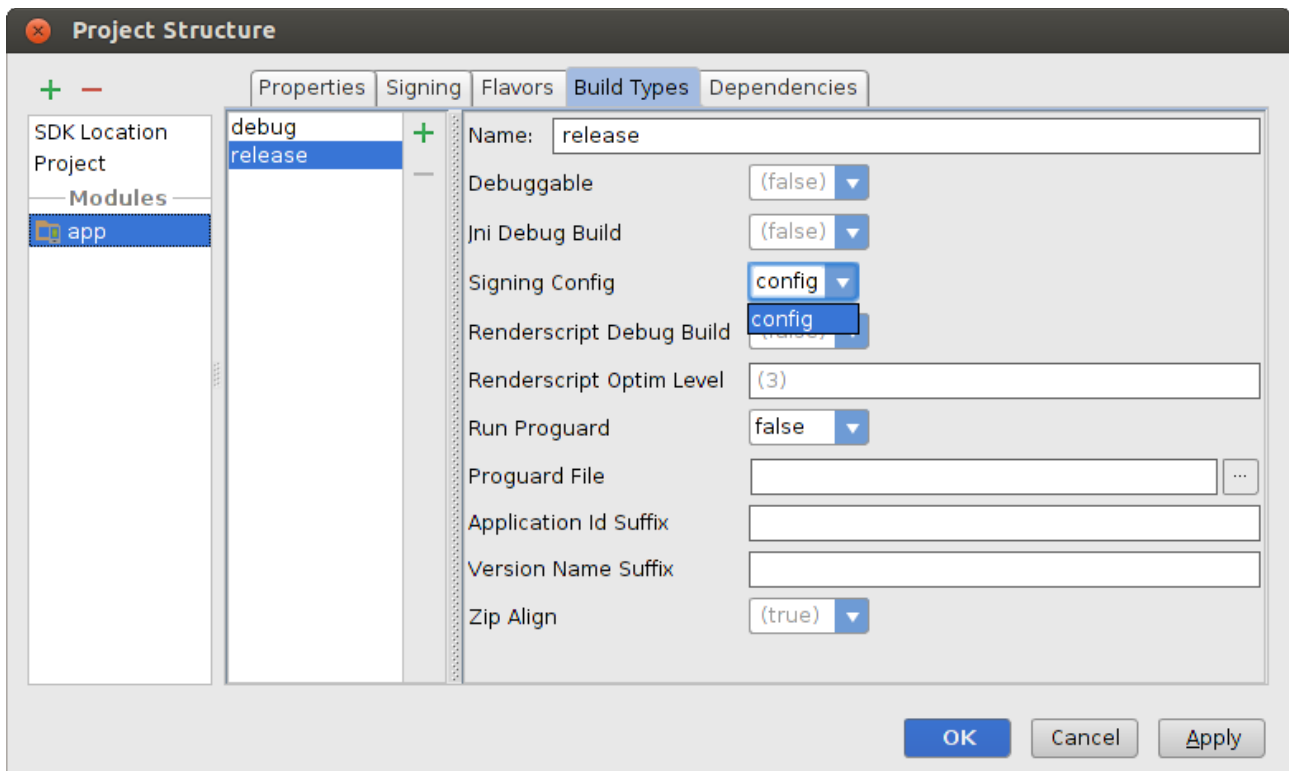
Automatically Signing Your App

In Android Studio, you can configure your project to sign your release APK automatically during the build process:

1. On the project browser, right click on your app and select **Open Module Settings**.
2. On the *Project Structure* window, select your app's module under *Modules*.
3. Click on the **Signing** tab.
4. Select your keystore file, enter a name for this signing configuration (as you may create more than one), and enter the required information.



5. Click on the **Build Types** tab.
6. Select the **release** build.
7. Under *Signing Config*, select the signing configuration you just created.
8. Click ok



Signing Your App Manually

You do not need Android Studio to sign your app. You can sign your app from the command line using standard tools from the Android SDK and the JDK. To sign an app in release mode from the command line:

1. Generate a private key using [keytool](#). For example:

```
$ keytool -genkey -v -keystore my-release-key.keystore  
-alias alias_name -keyalg RSA -keysize 2048 -validity 10000
```

This example prompts you for passwords for the keystore and key, and to provide the Distinguished Name fields for your key. It then generates the keystore as a file called `my-release-key.keystore`. The keystore contains a single key, valid for 10000 days. The alias is a name that you will use later when signing your app.

2. Compile your app in release mode to obtain an unsigned APK.

3. Sign your app with your private key using [jarsigner](#):

```
$ jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1  
-keystore my-release-key.keystore my_application.apk alias_name
```

This example prompts you for passwords for the keystore and key. It then modifies the APK in-place to sign it. Note that you can sign an APK multiple times with different keys.

4. Verify that your APK is signed. For example:

```
$ jarsigner -verify -verbose -certs my_application.apk
```

5.Align the final APK package using [zipalign](#).

```
$ zipalign -v 4 your_project_name-unaligned.apk your_project_name.apk
```

[zipalign](#) ensures that all uncompressed data starts with a particular byte alignment relative to the start of the file, which reduces the amount of RAM consumed by an app.

Get Started with Publishing

Start publishing on Google Play in minutes by:

- Registering for a Google Play publisher account
- Setting up a Google payments merchant account, if you will sell apps or in-app products.
- Exploring the [Google Play Developer Console](#) and publishing tools.

Register for a Publisher Account

1.Visit the [Google Play Developer Console](#).

2.Enter basic information about your **developer identity** — name, email address, and so on. You can modify this information later.

3.Read and accept the **Developer Distribution Agreement** for your country or region. Note that apps and store listings that you publish on Google Play must comply with the Developer Program Policies and US export law.

4.Pay a **\$25 USD registration fee** using Google payments. If you don't have a Google payments account, you can quickly set one up during the process.

5.When your registration is verified, you'll be notified at the email address you entered during registration.

Tips

- You need a Google account to register. You can create one during the process.
- If you are an organization, consider registering a new Google account rather than using a personal account.
- Review the [developer countries](#) and [merchant countries](#) where you can distribute and sell apps.

Set Up a Google Payments Merchant Account

If you want to sell priced apps, in-app products, or subscriptions, you'll need a Google payments merchant account. You can set one up at any time, but first review the list of [merchant countries](#).

To set up a Google payments merchant account:

1. **Sign in** to your Google Play Developer Console at <https://play.google.com/apps/publish/>.

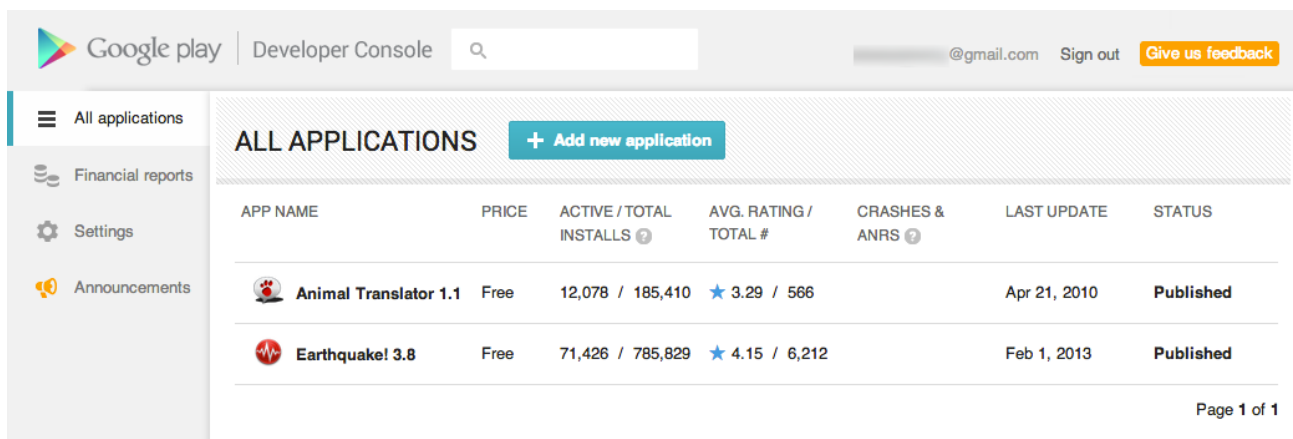
2. Open **Financial reports**  on the side navigation.

3. Click **Setup a Merchant Account now**.

This takes you to the Google payments site; you'll need information about your business to complete this step.

Explore the Developer Console

When your registration is verified, you can sign in to your Developer Console, which is the home for your app publishing operations and tools on Google Play.



The screenshot shows the Google Play Developer Console interface. At the top, there's a header with the Google Play logo, 'Developer Console', a search bar, and user information like '@gmail.com', 'Sign out', and a 'Give us feedback' button. On the left, a sidebar contains navigation links: 'All applications', 'Financial reports', 'Settings', and 'Announcements'. The main content area is titled 'ALL APPLICATIONS' and includes a '+ Add new application' button. Below this is a table listing applications with columns for App Name, Price, Active / Total Installs, Avg. Rating / Total #, Crashes & ANRS, Last Update, and Status.

APP NAME	PRICE	ACTIVE / TOTAL INSTALLS	AVG. RATING / TOTAL #	CRASHES & ANRS	LAST UPDATE	STATUS
 Animal Translator 1.1	Free	12,078 / 185,410	★ 3.29 / 566		Apr 21, 2010	Published
 Earthquake! 3.8	Free	71,426 / 785,829	★ 4.15 / 6,212		Feb 1, 2013	Published

Page 1 of 1

You can publish all your application from here.